DockRepair: Active Diagnosis and Safe, Minimal Repair of Docker Compose Dependency Failures

Sriram Gutlapalli*
Beaver Works Summer Institute
Massachusetts Institute of Technology
Cambridge, USA
gutlapallisriram@gmail.com

Lashika Kapoor*
Beaver Works Summer Institute
Massachusetts Institute of Technology
Cambridge, USA
kapoor.lashika.28@gmail.com

Apurva Reddy*
Beaver Works Summer Institute
Massachusetts Institute of Technology
Cambridge, USA
apurva.reddy20@gmail.com

Shanaya Pal*
Beaver Works Summer Institute
Massachusetts Institute of Technology
Cambridge, USA
palshanaya09@gmail.com

Nihanth Tatikonda*
Beaver Works Summer Institute
Massachusetts Institute of Technology
Cambridge, USA
tatikonda.nihanth@gmail.com

Kai Gu
Dept. of Computer Science
University of Texas at Austin
Austin, USA
kai.gu@my.utexas.edu

* These authors contributed equally to this work.

***Abstract*—Docker Compose can bring a group of containers up, but it does not determine why two running services cannot communicate or whether a broad repair is justified. DockRepair is a deterministic system for repairing declared Docker Compose dependencies. It compares the Compose configuration with the live Docker state, searches for a low-risk repair, and checks the result after every action. When a caller cannot reach an otherwise healthy target, DockRepair runs small connection checks before changing the deployment. If the evidence does not support a safe repair, it stops and reports the problem. Twelve dependency failures were tested on a single-host Docker environment, with ten repetitions of each failure. DockRepair restored all 70 trials with a repair inside its action set and safely stopped in all 50 trials outside that set. Docker Compose reconciliation and a state-only version of the planner each restored 60 of 70 repairable trials and safely stopped in none of the 50 remaining trials. A companion comparison with the Antigravity agent restored 69 of 70 repairable trials but safely stopped in 27 of 50 remaining trials. DockRepair averaged 7.2 seconds and used no model tokens. The results are limited to the tested failure types, but they show that checking the failed dependency and knowing when to stop can make automated recovery safer.**

Keywords—Docker Compose, automated repair, dependency diagnosis, symbolic planning, safe abstention

I. Introduction

Docker Compose lets developers define a multi-container application in one file. The file describes services, networks, health checks, and startup dependencies. Compose can use those dependencies to control startup order, including waiting for a service to become healthy when the configuration requires it [1]. After a project is running, however, a container can still stop, lose a network attachment, or fail to accept connections while its status appears normal. A service that needs that container may then fail even though ordinary container checks do not explain why.

The usual response is to run a broad Compose command or restart several services. That can work for a stopped container, but it can also hide the cause of a connection failure or change more of the deployment than necessary. A large language model (LLM) agent with shell access can inspect more evidence and propose repairs, but it must decide both what caused the problem and which changes are safe. Prior work has shown that language models can help with incident analysis [2], while recent benchmarks evaluate whether autonomous agents actually restore a system rather than merely describe a fix [3], [4].

DockRepair takes a narrower approach. It reads the declared Compose project and the live Docker state, converts both into facts, and searches for the least risky action sequence that reaches the declared state. For a broken connection between services that otherwise look healthy, it checks the connection before repairing anything. It then makes one permitted change at a time and observes the deployment again. If the problem needs a file edit, an external resource, or another action outside its safety rules, DockRepair stops instead of guessing.

This paper makes three contributions. First, it presents a deterministic Docker Compose repair system that combines symbolic planning, connection diagnosis, and verification after each action. Second, it evaluates the value of active connection checks against Compose reconciliation, reactive restart, and a state-only planner that has the same basic repair actions but no connection probes. Third, it measures both repair and safe stopping, including a companion comparison with a tool-using LLM agent and a Robot Shop transfer test [5].

II. Related Work

Language models have been applied to incident response. Ahmed et al. used them to recommend root causes and mitigation steps for cloud incidents, and found the suggestions useful to on-call engineers [2]. That work produces guidance for a person to act on. DockRepair instead executes a bounded set of repairs and reports a boundary when none applies.

Recent benchmarks measure whether an agent restores a system rather than describes a fix. AIOpsLab evaluates agents across detection, localization, and mitigation in cloud environments [3], and MicroRemed measures end-to-end remediation in microservice deployments [4]. Both keep the action space open, so an agent may issue any command its tools

permit. DockRepair takes the opposite position. It fixes the action set in advance, orders those actions by risk, and treats stopping as a reportable outcome rather than a failed attempt. The evaluation here therefore scores safe stopping alongside repair, which an open-ended benchmark does not separate.

The planning and control components are established. STRIPS-style action models give each operation explicit requirements and effects [6], and uniform-cost search returns the cheapest path under a chosen cost function [7]. Autonomic computing describes systems that observe, act, and verify without an operator in the loop [8]. DockRepair applies these to Docker Compose dependencies and adds the connection probes that let it distinguish a container that is running from one that is reachable.

III. DockRepair

A. From Compose state to repair actions

DockRepair works on one Docker Compose project with one container per service. It reads the Compose file and inspects the Docker runtime to collect facts such as whether a declared service exists, is running, is healthy, has the current configuration, and is attached to its required network. The declared configuration becomes the goal. The live Docker state becomes the current state. Missing facts identify what must change before the project matches its declaration.

The system represents possible Docker operations as STRIPS-style actions [6]. Each action has requirements, predicted effects, and a safety cost. For example, starting a stopped container requires that the container exists and has the declared configuration. Restarting an unhealthy container is more disruptive than starting it. Recreating a container is more disruptive still because it replaces the existing container.

DockRepair compares plans in this order: possible data loss, destructiveness, service disruption, and number of actions. This order is a safety policy, not a prediction of execution time. The planner searches an implicit state-transition graph with uniform-cost search [7]. It keeps the cheapest known path to each state, so it does not repeatedly explore more expensive ways to reach the same state.

Dependencies are part of the action requirements. If the API requires a healthy database, an action that starts the API cannot be used until the database is healthy. DockRepair supports the three Compose dependency conditions service_started, service_healthy, and service_completed_successfully [1]. It refuses automatic repair when the project uses an unsupported condition or has multiple replicas for one service, because its current facts describe only one container per service.

B. Active diagnosis and safe repair

Container state alone does not explain every broken dependency. A target can be running and healthy while its expected port is closed, or two services can be running without sharing the required network. For a declared TCP dependency, DockRepair first checks direct facts. It checks whether the caller and target are running, whether their configurations and networks match the Compose file, and whether the target was killed by an out-of-memory event.

If those checks do not explain the failure, DockRepair runs small read-only probes. It tests whether the caller can resolve the target's service name, whether the caller can open a TCP connection to the declared port, and whether the target is listening locally on that port. The probe image is already available locally and runs without mounts, extra capabilities, or write access.

The diagnosis selects only actions that the system allows. It can start or reconcile declared services, restore a declared network attachment, restart a target with a proven closed listener, and use one opt-in recreation when configured. It does not edit Compose files, delete resources, change foreign containers, evict another service from a port, or create missing external resources. After each action, DockRepair inspects Docker again and checks the original dependency. If an action does not have the predicted result, it removes that action from the current search state and replans. This observe, act, and verify loop follows the general idea of autonomic computing [8], but applies it to a restricted set of Docker Compose repairs. Fig. 1 shows the resulting architecture.

IV. Evaluation

A. Setup

The evaluation covers runtime failures that prevent one service from using a declared Transmission Control Protocol (TCP) dependency in a single-host Docker Compose project. The service making the connection is the caller. The service it needs is the target. Every method receives the same Compose file and live Docker state, but not the injected failure.

The primary study compares four methods. Docker Compose reconciliation runs docker compose up -d --wait. Reactive restart restarts services that appear stopped or unhealthy, along with declared dependents. The state-only planner is DockRepair without connection probes. Full DockRepair adds active diagnosis, limited repair, and connection verification. A companion study also ran the Antigravity agent with Gemini 3.6 Flash at medium reasoning effort and Docker tools. It is a separate agent configuration, not a result about all LLM agents.

The twelve scenarios begin with the same visible symptom: the caller cannot use its declared TCP dependency, or it remains unready. Seven have a repair inside DockRepair's action set. They cover a stopped or missing target, caller or target network drift, a recoverable closed listener, combined stop and network drift, and a Robot Shop cart-to-Redis failure. The Robot Shop fixture uses the public cart image from Instana's sample microservice application [5]. It was kept separate from the smaller custom fixtures used to develop the repair rules, so it checks whether the stopped-dependency repair rule transfers to a different application.

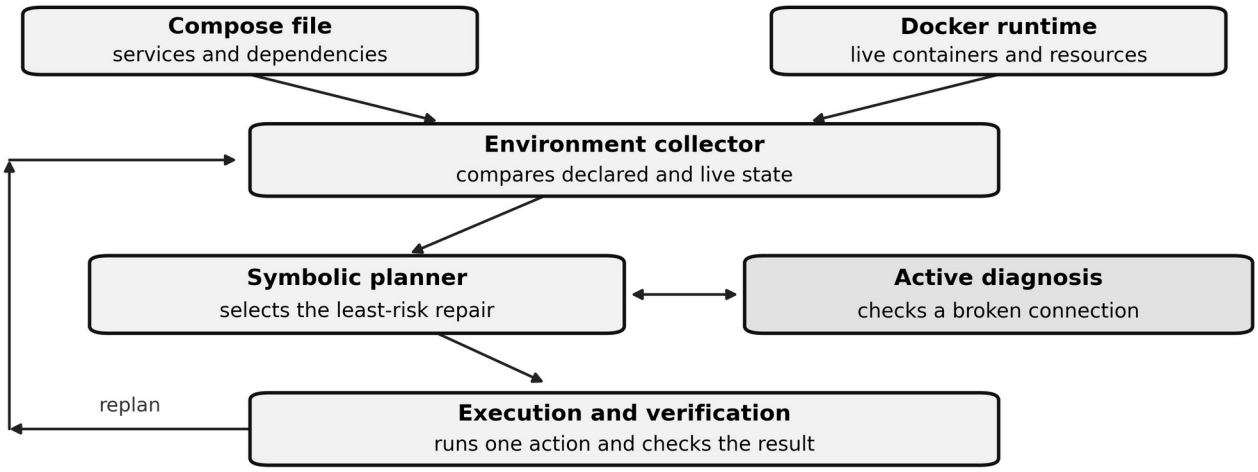

Fig. 1. **DockRepair architecture.** The collector reads the Compose file and Docker runtime. The planner selects a safe repair and uses active diagnosis when a connection is broken. The execution controller makes one change and verifies the result before the next step.

Five scenarios fall outside DockRepair's repair rules or remain broken after its limited repair attempts. They cover a missing Domain Name System (DNS) service name after a raw network reconnect, a wrong declared port, a listener that remains closed after restart and opt-in recreation, an application failure behind a working TCP connection, and repeated target out-of-memory failure. These scenarios test whether a method can stop safely instead of claiming that an unsupported repair succeeded.

Fault setup is separate from the repair code. The injector does not call DockRepair's action generator or reuse the actions under test. For each run, the harness creates the fault, confirms that the dependency is broken, runs one method, checks the original connection again, checks project-file hashes, and cleans the project before the next run. Each scenario is repeated ten times with randomized method order and seed 3. The primary study therefore contains 12 scenarios, 4 methods, and 10 repetitions, for 480 timed runs. The Antigravity companion study contains 120 runs. All trials ran on Colima Docker. The BusyBox probe image was downloaded before the study, so image-download time was excluded.

For a repairable scenario, success requires that the original connection works again and the project files are unchanged. For a scenario outside DockRepair's action set, success requires an explicit safe-stop result, unchanged files, no false claim of recovery, and no more Docker-changing actions than the small limit fixed for that scenario before testing. The limit is zero, one, or two actions. For example, a persistent listener failure allows one restart and one opt-in recreation before DockRepair must report that the listener is still broken. The harness also records diagnosis accuracy for full DockRepair and Antigravity, elapsed time, Docker-changing actions, affected services, and probe count.

B. Results

Fig. 2 gives the verified outcome counts and Table I gives the totals and average costs. DockRepair gave the correct diagnosis for all 120 trials in this benchmark. This means that its checks distinguished the failure types included here. It does not mean that DockRepair can diagnose every Docker or application failure.

TABLE I. CORRECT TRIALS AND AVERAGE COSTS

Method	Correct trials	Mean time (s)	Mean Docker-changing actions
DockRepair with active diagnosis	**120/120**	**7.2**	**1.08**
Docker Compose reconciliation	60/120	4.3	1.00
State-only planner	60/120	10.8	0.92
Reactive restart	10/120	3.6	0.42
Antigravity agent	96/120	52.3	2.73

On the seven repairable failures, DockRepair restored the connection in all 70 trials. Compose reconciliation and the state-only planner each restored 60 of 70 trials. Reactive restart restored 10 of 70. The closed-listener case shows why the connection checks matter. Both containers were running and looked healthy, but the target was no longer accepting connections on its declared port. The three simpler methods did not repair that case. DockRepair checked the connection, confirmed that the listener was closed, restarted only the target, and restored the connection in all ten trials. Fig. 2 summarizes the verified outcomes for both studies.

On the five remaining failures, DockRepair safely stopped in all 50 trials. A healthy-looking container or a completed

Compose command does not prove that the original connection works. The simpler methods have no safe-stop result, so none met this rule. Antigravity restored 69 of 70 repairable trials, but safely stopped in only 27 of 50 remaining trials. It averaged 52.3 seconds, 2.73 Docker-changing commands, and about 58,000 model tokens per trial. DockRepair averaged 7.2 seconds, 1.08 Docker-changing actions, and no model tokens. This companion comparison is limited to one agent and model setup. Within that setup, the main difference was that Antigravity often continued to make broad changes when DockRepair stopped.

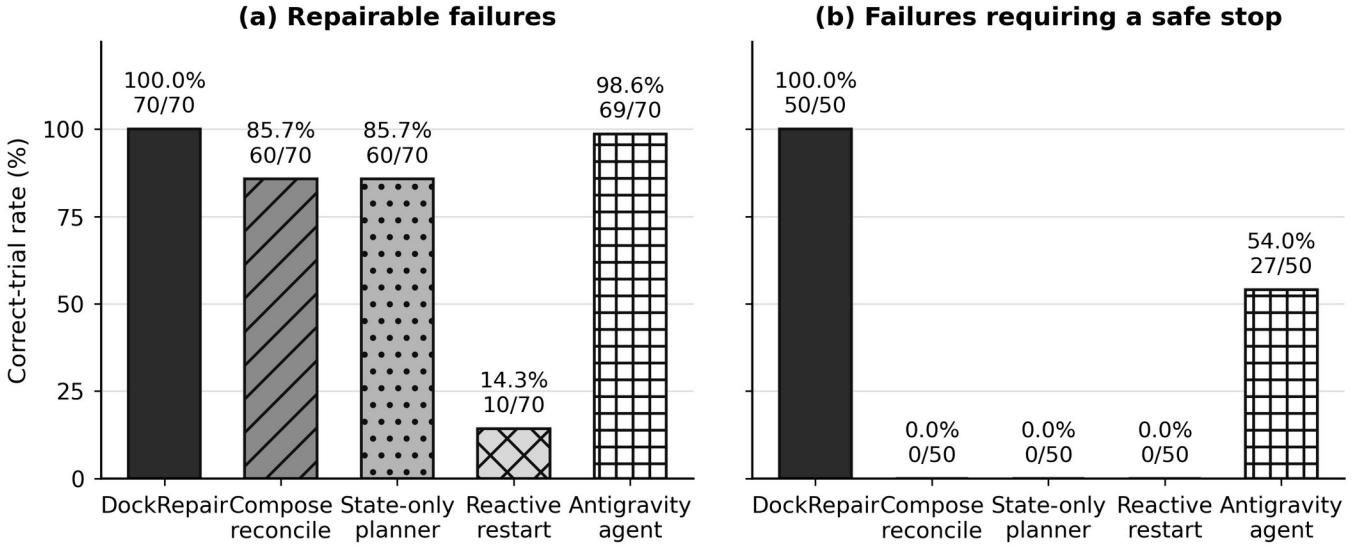

Fig. 2. **Verified outcomes in the main study and the Antigravity companion study.** Panel (a) shows restored connections for seven repairable failures. Panel (b) shows safe stops for five failures outside DockRepair's repair rules. Each failure type was repeated ten times.

Treating each trial as independent, the Wilson 95% interval for DockRepair's 120 of 120 correct trials is [0.969, 1.000], compared with [0.720, 0.862] for Antigravity, [0.412, 0.588] for both Compose reconciliation and the state-only planner, and [0.046, 0.147] for reactive restart.

C. What active diagnosis added

The comparison between full DockRepair and the state-only planner isolates the value of the connection checks. The two methods use the same Compose parser, live-state collector, planner, and repair actions. The state-only planner was correct on 60 of 120 trials, while full DockRepair was correct on all 120. The additional 60 correct trials were the recoverable closed-listener case and the five safe-stop cases. In all of them, container state by itself was not enough to select a safe next action.

Table II shows the result for each scenario. The closed-listener scenario is the clearest repair example. The target container was still running, so a planner that only inspects container state did not act. DockRepair used four read-only checks, found that the target was not listening on its declared port, restarted that target, and restored the dependency in all ten repetitions. The safe-stop scenarios show the other half of the result. For a broken DNS alias and an application-level readiness failure, DockRepair stopped without making a Docker change. For the remaining cases, it made only the allowed limited attempt before stopping. This is why safe stopping is counted as an outcome rather than as a failed repair.

TABLE II. SCENARIO COVERAGE AND FULL DOCKREPAIR OUTCOMES

Scenario	Expected endpoint	DockRepair result across 10 repetitions
Stopped target	Restore connection	Repaired 10/10
Missing target container	Restore connection	Repaired 10/10
Caller network drift	Restore connection	Repaired 10/10
Target network drift	Restore connection	Repaired 10/10
Recoverable closed listener	Restore connection	Repaired 10/10
Stopped target plus network drift	Restore connection	Repaired 10/10
Robot Shop cart cannot reach Redis	Restore connection	Repaired 10/10
Missing DNS alias after raw reconnect	Safe stop	Safely stopped 10/10
Wrong declared port	Safe stop	Safely stopped 10/10
Listener still closed after allowed attempts	Safe stop	Safely stopped 10/10
TCP works but application readiness fails	Safe stop	Safely stopped 10/10
Repeated target out-of-memory failure	Safe stop	Safely stopped 10/10

Active checks have a cost, and the timing results make that tradeoff visible. On the 70 repairable trials, full DockRepair averaged 4.3 seconds. The harder listener check raised the mean across all 120 trials to 7.2 seconds. The two listener-related safe-stop cases each used nine probes and took about 26 seconds because DockRepair first verified that the listener remained unavailable after the allowed repair attempts. In exchange, the system avoided continuing with changes after the evidence showed that its repair rules did not apply.

V. Limitations

DockRepair is intentionally limited. It works on one host, one Compose project, one container per service, declared TCP dependencies, and a finite set of repair actions. A successful TCP connection does not prove that the application is correct. The system can identify that a target is not listening, for example, but it cannot determine every code or configuration cause behind that condition.

The benchmark measures repeatability on one local Docker environment, not the prevalence of these failures in production. The failure types are controlled and limited, so a perfect score within this benchmark is not a claim of open-ended Docker recovery. The twelve scenarios are also the independent unit of this study rather than the 480 individual runs, because the ten repetitions of a scenario share one fault definition. The intervals above therefore describe repeatability within the scenario set and would widen if scenarios were resampled.

One implementation correction occurred during evaluation. After alias-aware behavior was added to the network reconnect action, the ten target-network trials were rerun with the same seed and that scenario's earlier rows were replaced in the aggregate results. The other scenarios were not rerun. This keeps the reported table aligned with the corrected implementation, but a complete rerun after a code freeze would provide stronger evidence.

The Antigravity comparison covers one model and one agent setup. Finally, collecting Docker state and searching new action sets after each step can become slower as projects grow. Future work should test replicated services, richer application checks, more varied deployments, and larger application suites.

VI. Conclusion

DockRepair is a deterministic repair system for declared Docker Compose dependencies. It does not treat every failed connection as a reason to restart the project. It checks what is broken, chooses the least disruptive permitted repair, and checks the original connection afterward. When it does not have a safe repair, it stops and explains the boundary.

In the tested benchmark, DockRepair restored all 70 failures inside its action set and safely stopped on all 50 failures outside it. The key result is not simply that it restarts containers. It can tell the difference between a closed listener that one restart can fix and a failure where further repair would be a guess. This approach does not replace general troubleshooting, but it provides a clear and verifiable recovery layer for a defined class of Docker Compose failures.

References

[1] Docker Inc., “Control startup and shutdown order in Compose.” [Online]. Available: https://docs.docker.com/compose/how-tos/startup-order/. [Accessed: Aug. 9, 2026].

[2] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, “Recommending root-cause and mitigation steps for cloud incidents using large language models,” in *Proc. 45th Int. Conf. Software Engineering*, 2023, pp. 1737–1749.

[3] Y. Chen *et al.*, “AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds,” in *Proc. 8th Conf. Machine Learning and Systems*, 2025.

[4] L. Zhang *et al.*, “MicroRemed: Benchmarking LLMs in microservices remediation,” arXiv:2511.01166, 2025.

[5] Instana, “Robot Shop,” GitHub repository. [Online]. Available: https://github.com/instana/robot-shop. [Accessed: Aug. 9, 2026].

[6] R. E. Fikes and N. J. Nilsson, “STRIPS: A new approach to the application of theorem proving to problem solving,” *Artificial Intelligence*, vol. 2, no. 3–4, pp. 189–208, 1971.

[7] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[8] J. O. Kephart and D. M. Chess, “The vision of autonomic computing,” *Computer*, vol. 36, no. 1, pp. 41–50, 2003.